

Sebastian David Albu

Extended Cryptography Task Generator: Creating new Differential Cryptanalysis and Elliptic Curve Tasks

BACHELOR'S THESIS

Bachelor's degree programme: Computer Science

Supervisors

Marcel Nageler

Maria Eichlseder

Institute of Applied Information Processing and Communications
Graz University of Technology

Graz, July 2024

Abstract

Security is, without a doubt, one of the most critical aspects of today's society. So, it should be no surprise that the University of Technology in Graz has a lot of research and courses in cybersecurity. Among those courses, there are several in which cryptography is taught. Cryptography is one of the fundamental aspects of security. Therefore, it is essential for every student interested in information security. To ease the life of lecturers, we want to create a tool generator that helps them to generate cryptography tasks for the course 'Cryptography' [IAI24b], taught at the Institute of Applied Information and Communications at the Graz University of Technology [IAI24a]. Since coming up with new tasks for cryptographical courses is indeed challenging and time-consuming for lecturers, the main idea of this thesis is to analyze if and how one could automate this process. This thesis covers the challenges and obstacles of finding new correct tasks for elliptic curve cryptography and differential cryptanalysis. This thesis will demonstrate how the new task generator tool overcomes these obstacles and can generate new tasks and solutions using Python, SageMath, Z3, and $\text{\LaTeX}$. With this new advanced tool, lecturers can directly input the tasks they want to generate, followed by the wanted parameters needed in a terminal interface, and get a PDF and $\text{\LaTeX}$ source file. Suppose the given inputs or randomly chosen input values do not provide satisfying results. In that case, the users can also adapt the input parameters without losing the already computed tasks and generate as many tasks as they wish until they find one with the desired outcome. This allows for a fast and user-friendly environment.

Keywords: Security · Teaching · Exercises · Cryptography · Differential Cryptanalysis · Differential Distribution Table (DDT) · Substitution Permutation Network (SPN) · Elliptic Curves · ElGamal Encryption ·

Contents

1	Introduction	4
2	Background	6
2.1	Elliptic Curve Terminology	6
2.1.1	Fundamentals: Finite Fields and Binary Fields	6
2.1.2	Elliptic Curve Groups	8
2.1.3	Security Aspects of Elliptic Curves	9
2.2	Differential Cryptanalysis Terminology	10
2.2.1	Differential Distribution Table (DDT)	10
2.2.2	Substitution Permutation Networks (SPN)	12
2.2.3	Feistel Networks (FN)	13
3	Elliptic Curve Cryptography - Tasks and Implementation	15
3.1	Fundamentals: Finite Fields and Binary Fields	15
3.2	Elliptic Curve Groups	16
3.3	ElGamal encryption for Elliptic Curves	19
3.4	Security Aspects of Elliptic Curves	20
4	Differential Cryptanalysis - Tasks and Implementation	23
4.1	Differential Distribution Table (DDT)	23
4.2	Simple 1-Round Toy Cipher Cryptanalysis	24
4.3	Substitution Permutation Network (SPN) Cryptanalysis	26
4.4	Feistel Network (FN) Cryptanalysis	37
5	Conclusion	42
	Bibliography	43

1 Introduction

Lecturers at universities tend to face a common problem: Generating new and correct exercises and exams takes considerable effort and time. This time can better be invested in research. It would be much easier if there were automated tools that generate tasks and their solutions automatically. This is precisely what we try to solve within the work of this thesis.

Related Work. We already have a working tool for generating cryptographical tasks for primality testing, factorization, RSA, and stream ciphers, implemented by Romana Gruber [Gru20]. This tool is also used in the scope of the “Cryptography” [IAI24b] course.

Another example of a tool used to generate examples for the examination of students is used in the course “Operating Systems” [IAI24c]. The generator for the operating systems course is also publicly available to students [IAI24d].

Goal. This thesis aims to extend the cryptography task generator by Romana Gruber [Gru20] and introduce differential cryptanalysis and elliptic curve cryptography tasks. The plan is to adjust the existing tool and expand it with more functionalities, such as the possibility to directly enter everything in the terminal with the help of the Python library “Click”, allowing for a faster user interface, like the already existing GUI.

The user gets a list of all possible tasks and can select the desired one. After selecting one specific task, the user also has some randomly selected default values suggested directly and can hit the enter button a few times and get a valid $\text{\LaTeX}$ string stored in the generator. The user can also see in an overview which tasks they have generated and the number of generated sub-tasks. The tasks can either be about *Differential Cryptanalysis* or *Elliptic Curve Cryptography*, covering both *symmetric* and *asymmetric* cryptography tasks.

The tasks about differential cryptanalysis roughly consist of filling out a *Differential Distribution Table* (DDT) where rows and columns are missing, reducing the key space in a simple one-round block cipher, differential cryptanalysis on a *Feistel cipher* and most importantly, the differential cryptanalysis on a *Substitution Permutation Network* (SPN) cipher.

The tasks about elliptic curve cryptography involve the introduction to *Finite Fields*, the calculation of the group points inside an *Elliptic Curve Group*, applying the well-known *ElGamal* encryption on elliptic curves, and discussing and analyzing security properties of elliptic curves.

Procedure. For the creation and solution of these mentioned tasks, we mostly rely on the materials given in the course “Cryptography” [IAI24b] and also focus on work from Heys, Howard M [Hey02] and Christina Boura [Bou18] for the tasks about differential cryptanalysis, and on the work from Amara, Moncef and Siad, Amar [AS11] and Menezes, Alfred [Men01] for the tasks about elliptic curves. We mainly use the descriptions already given in the course for the task description procedures.

The tool also generates a $\text{\LaTeX}$ source file with a section for the solution, which can be left in or commented out. It is then possible to go inside the generated $\text{\LaTeX}$ file and (un)comment the line that allows the displaying of the solutions. With this, it is possible to adjust the assignment dates in the $\text{\LaTeX}$ file and select the wanted tasks before generating the PDF file.

A sketch of how lecturers can use the tool is demonstrated in Figure 1.1.

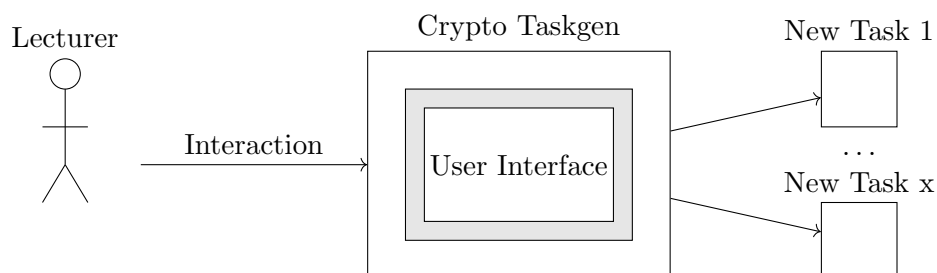


Figure 1.1: Very simplified sketched interaction of user and the task generator

Such tools and ideas always seem way too much like a blessing. Indeed, we will encounter many pitfalls in this thesis while we break down the creation of the task generator and its needed algorithms.

Outline. First, in chapter 2, we explain the needed mathematical and cryptographical background regarding elliptic curves and differential cryptanalysis. Then, in chapter 3, we present the elliptic curves task for students and how the task generation procedure works. In chapter 4, we present the tasks regarding differential cryptanalysis and the task generation procedure. Finally, in chapter 5, we discuss our findings and directions for future work.

2 Background

Before it is possible to go over the generated tasks, we need to address some fundamentals. First, we discuss the definitions for *Elliptic Curves* in section 2.1. Afterward we cover the basics of *Differential Cryptanalysis* in section 2.2.

2.1 Elliptic Curve Terminology

Elliptic curves are fundamental to modern-day cryptography. They come with many properties that are well-suited for cryptographic purposes. This chapter reviews the mathematical aspects of elliptic curves and covers some of their use cases and security properties.

2.1.1 Fundamentals: Finite Fields and Binary Fields

First, the definitions of groups and fields are needed to understand elliptic curve groups. For this subsection, we refer to the book by Lidl et al. [LN94] unless stated otherwise.

Definition. A *group* is a set G together with a binary operation $*$: $G \times G \rightarrow G$, such that the following properties are satisfied:

1. Closure: $\forall a, b \in G, a * b \in G$. In other words, the result of the group operation applied to any two elements in the group is always another element of the group.
2. Associativity: $\forall a, b, c \in G$, the equation $a * (b * c) = (a * b) * c$ holds.
3. Identity Element: $\exists e \in G$ such that $\forall a \in G, e * a = a * e = a$.
4. Inverses: $\forall a \in G, \exists a' \in G$ (called the inverse of a) such that $a * a' = a' * a = e$, where e is the identity element.

Definition. A *finite cyclic group* is a group where all elements can be obtained by repeatedly applying the group operation starting from a particular element. More formally, a group G is finite when there is a finite number of elements in the group. The group is cyclic if there exists an element $g \in G$ such that every element of G can be written as g^n for some integer n .

Definition. This element g is then the *generator* of the finite cyclic group. In other words, g generates G , denoted as $G = \langle g \rangle$, meaning that by repeatedly applying the group operation to g , all elements of G can be reached.

Definition. The *order* of a group element is the number of applied group operations until the group element reaches the neutral element. For the generator g of the finite cyclic group G , it also holds that the *order* of $g = |G|$, meaning $|g| = |G|$.

Definition. A *field* $\langle S, +, \cdot \rangle$ is a set S with two operations $+$ and $\cdot$ such that:

1. **Additive group:** $\langle S, + \rangle$ is an Abelian group with neutral element 0.
2. **Multiplicative group:** $\langle S \setminus \{0\}, \cdot \rangle$ is an Abelian group with neutral element 1.
3. **Distributive:** $c \cdot (a + b) = c \cdot a + c \cdot b$.

Definition. A *finite field* (“Galois Field”) is a field with a finite number of elements. A finite field of type $\mathbb{F}_{p^n}$, where p is prime and $n \geq 1$, consists of vectors of length n with entries from $\mathbb{Z}_p$. The field is constructed using *irreducible polynomials* modulo p . “Irreducible” in this sense means that it cannot be factored into smaller polynomials over $\mathbb{F}_p$, similar to primes. Each p^n size field is unique, although multiple constructions can result in fields of the same size but potentially different representations.

Definition. The *addition and multiplication inside a binary field* are then inspired by rules from complex numbers $\mathbb{C}$ with $a \cdot i + b$, with the imaginary unit i . In a field it then holds that in $\mathbb{F}_{p^n}$, $p = 2$ and $n \geq 1$. We label the n vector entries $a_{n-1}, a_{n-2}, \dots, a_1, a_0$ and interpret them as coefficients of a formal polynomial of degree $d \leq n - 1$

$$f(x) = a_{n-1}x^{n-1} + a_{n-2}x^{n-2} + \dots + a_1x + a_0.$$

The addition is an XOR, denoted as $\oplus$, between the coefficients of the same power.

For multiplication, we will reduce the result to degree $n - 1$ with the modulus being an irreducible polynomial. The multiplication itself works similarly to regular multiplication over $\mathbb{C}$, and the procedure is summarized in Algorithm 1.

Example. Assuming the multiplication given $(x^7 + x^6 + x^4 + x^2 + x + 1) \cdot (x^2 + 1)$ over the irreducible polynomial $(x^8 + x^4 + x^3 + x + 1)$, the first intermediate result would happen when multiplying $(x^7 + x^6 + x^4 + x^2 + x + 1) \cdot (x^2)$. The standard multiplication rule applies, resulting in $(x^9 + x^8 + x^6 + x^4 + x^3 + x^2)$. The last term becomes x^2 because $1 = x^0$. Now, we also need to multiply the original polynomial with 1. This is then just adding the original polynomial to the intermediate result. This results in $(x^9 + x^8 + x^6 + x^4 + x^3 + x^2) + (x^7 + x^6 + x^4 + x^2 + x + 1)$, which is then $(x^9 + x^8 + x^7 + x^3 + x + 1)$ after adding them together. However, the degree of this intermediate result is nine; therefore, we need to reduce it to match the degree of the irreducible polynomial. We will check for the degree difference between the intermediate results and the irreducible polynomial to do this. We store this difference in d and calculate an irreducible polynomial multiplied by x^d . In our case $d = 1$ and we multiply $(x^8 + x^4 + x^3 + x + 1) \cdot (x^1)$, resulting in $(x^9 + x^5 + x^4 + x^2 + x)$. We perform an XOR between the two polynomials: $(x^9 + x^8 + x^7 + x^3 + x + 1) \oplus (x^9 + x^5 + x^4 + x^2 + x)$,

resulting in $(x^8 + x^7 + x^5 + x^4 + x^3 + x^2 + 1)$. We must again divide with the irreducible polynomial multiplied by x^d since the degree equals p . So, $d = 0$ and we have an XOR between the second intermediate result and the regular irreducible polynomial $(x^8 + x^7 + x^5 + x^4 + x^3 + x^2 + 1) \oplus (x^8 + x^4 + x^3 + x + 1)$ resulting in $(x^7 + x^5 + x^2 + x)$, which is the final result.

Algorithm 1 Binary Field Multiplication

```

1: procedure BINARYFIELDMULTIPLY( $a, b, f$ )
2:   Let  $f(x)$  be the irreducible polynomial and  $a, b$  the polynomials to be multiplied
3:   Initialize an empty list  $result$ 
4:   for  $i = 0$  to  $\text{degree}(a) + \text{degree}(b)$  do
5:      $result[i] \leftarrow 0$ 
6:   end for
7:   for  $i = 0$  to  $\text{degree}(a)$  do
8:     for  $j = 0$  to  $\text{degree}(b)$  do
9:        $result[i + j] \leftarrow result[i + j] \oplus (a[i] \cdot b[j])$ 
10:    end for
11:  end for
12:  while  $\text{degree}(result) \geq \text{degree}(f)$  do
13:    Find  $d = \text{degree}(result) - \text{degree}(f)$ 
14:    Calculate  $f(x) \cdot x^d$ 
15:     $result \leftarrow result \oplus (f(x) \cdot x^d)$ 
16:  end while
17:  return  $result$ 
18: end procedure

```

2.1.2 Elliptic Curve Groups

With all the needed definitions for groups and finite fields, we now cover the definitions of elliptic curve groups. For this section, we refer to the work of Amara, Moncef, and Siad, Amar [AS11] unless stated otherwise.

Defintion. An *algebraic curve* is a geometric object in two variables defined by an algebraic equation. Specifically, an algebraic curve is the set of points satisfying a polynomial equation of the form $F(x, y) = 0$, where F is a polynomial in x and y . Elliptic curves are a particular type of algebraic curve defined by equations of degree 3. There are several ways to define the implicit polynomial equation, resulting in alternating types of elliptic curves. For our tasks, however, we want to only work with elliptic curve groups over $\mathbb{F}_{2^p}$. The implicit equation is called the *Weierstrass equation* and is given as

$$y^2 = (x^3 + ax + b) \pmod{p},$$

where a and b are integers $\pmod{p}$ in our examples.

Definition. A *point* P is defined via its coordinates (X_P, Y_P) . Since the elliptic curve is defined over $\mathbb{F}_{2^p}$, it must hold that every point P in the group must have its inverse $-P$. The inverse $-P$ is then defined via $(X_P, -Y_P)$, whereas the calculation of $-Y_P$ is nothing but the modular inverse of Y_P . In every elliptic curve group, there exists the neutral element $\mathcal{O}$, which is the “point at infinity” $(0, \infty)$. This point needs to exist. Otherwise, the group would not be closed, and calculations for adding $P + (-P)$ would result in a point outside the group when applying the *Chord Rule*, which is needed for point addition. For addition of the point to itself ($2P$), there is the *Tangent Rule*. Both the Tangent and the Chord rule are taken from the publication by Haoru Liu [Hao11].

Definition. A *generator* G of an elliptic curve group is a special point on the curve such that every other point in the group can be expressed as an integer multiple of G . Formally, if E is an elliptic curve defined over the finite field $\mathbb{F}_{2^p}$, and G is a generator of the group of points on E , then for any point P in this group, there exists an integer n such that $P = nG$. The smallest positive integer n for which $nG = \mathcal{O}$ is called the *order* of G . The existence of a generator is crucial for the group structure, as it ensures that the group is cyclic, a fundamental property used in various cryptographic applications. Choosing a generator point G can significantly impact the security and efficiency of elliptic curve-based cryptographic systems.

2.1.3 Security Aspects of Elliptic Curves

Not every algebraic curve that solves the Weierstrass equation can be used as a secure elliptic curve. Some criteria can even let the results of the implicit equation not be an elliptic curve at all. The following methods can be used to check the security aspects of an elliptic curve.

Definition. The *trace of Frobenius* plays a significant role in choosing the correct parameters, as also seen in the publication by Bos *et. al* [BCLN16]. For a curve E over $\mathbb{F}_q$ with $|E| = n$, the trace of Frobenius t is defined by

$$n = q + 1 - t, \quad (2.1)$$

where q is given by the underlying field $\mathbb{Z}_q$ and should be equal to p^m , with p being a prime number and $m \geq 1$. This equation then helps to find *anomalous* and *supersingular* curves. We can rearrange Equation 2.1 to solve for t . If the result gives us $t = 1$, we have found an anomalous curve unsuitable for elliptic curves, especially if $m = 1$. The resulting point order is small, allowing for easier ECDLP computation. A supersingular elliptic curve has a special property, allowing attackers to bypass the mathematical strengths that secure elliptic curves, as seen in [BCLN16]. The exact security risks, however, are just briefly mentioned in this course, and therefore, students only need to be able to find the weakness and roughly explain it. For a more in-depth explanation and derivation of the trace of Frobenius and its application, we refer to the publication by Loren D. Olson [Ols79].

Definition. The *discriminant* can be used to check if the solution of the Weierstrass equation ($y^2 = x^3 + ax + b$) is a valid elliptic curve. The discriminant for this type of curve is given as

$$\Delta = -16 \cdot (4a^3 + 27b^2) \pmod{q}.$$

If $\Delta = 0$, it means that the curve is *singular*, and we do not even have an elliptic curve. In the case of a singular curve, SageMath will also fail to construct an elliptic curve and throw assertion errors. This case of the problematic curve also gives us some problems when calculating the order, as will be discussed in paragraph 3.4.

2.2 Differential Cryptanalysis Terminology

Linear and differential cryptanalysis are well-known attacks to symmetric-key block ciphers. Differential cryptanalysis dates back to the early 1990s. Biham and Shamir (co-inventor of RSA [RSA78]) had ideas in their publication [BS93] to break the DES encryption scheme by applying the techniques now found in differential cryptanalysis. The core idea of differential cryptanalysis is analyzing how the input differences affect output differences. So, there are two plaintext messages, and an XOR operation is performed on them. This resulting difference is called the *input difference* and denoted as ΔI or the difference of the plaintext messages ΔM .

We have two distinct ciphertexts. By XOR-ing these, we again get a difference, now denoted as the *output difference* ΔO , or the difference of the ciphertexts ΔC . The concept of differential cryptanalysis now boils down to analyzing how we can determine the key or a reduced keyspace (the number of possible keys) by examining the influences of ΔM to ΔC .

For this entire subsection, we heavily rely on the publication by Heys [Hey02] unless stated otherwise.

2.2.1 Differential Distribution Table (DDT)

The differential distribution table is an essential building block for differential cryptanalysis. In this chapter, we will first review the needed background of the DDT and how it helps to analyze S-boxes $\mathcal{S}$, commonly used in ciphers.

In all of our examples we will have an S-box $\mathcal{S}$.

Definition. Simplified, the *S-box* $\mathcal{S}$ is a non-linear look-up table (LUT) that provides a direct mapping between input values and corresponding output values.

The idea is to have the S-box be as non-linear as possible to cause *confusion*, which dates back to Shannon's principle [Sha49]. So, for example, having every number from 0 to $\mathbf{f}$ just being set by an offset by one is not ideal and is easily breakable. Table 2.1 is a bad example of an S-box.

Table 2.1: Linear LUT, bad example of an S-Box $\mathcal{S}$

x	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
$\mathcal{S}(x)$	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f	0

Table 2.2: Nonlinear LUT, better example of an S-Box $\mathcal{S}$

x	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
$\mathcal{S}(x)$	6	2	3	a	0	e	4	7	8	f	c	b	1	5	9	d

Even people unfamiliar with cryptography will easily spot the pattern in Table 2.1, and differential cryptanalysis becomes significantly easier. We would instead aim for a random-looking LUT with no easily distinguishable pattern, as in Table 2.2.

However, developing new secure S-boxes is difficult, so we aim instead to create *affine equivalent* S-boxes. This will be further explained in paragraph 4.1.

Definition. The *differential distribution table* (DDT) is constructed by calculating all possible output differences for every possible input difference of an S-box $\mathcal{S}$. The origins of the DDT date back to Biham and Shamir [BS93], where it was originally defined as the *difference distribution table*. It involves solving the equation

$$\mathcal{S}(x \oplus \Delta u) \oplus \mathcal{S}(x) = \Delta v.$$

In simpler terms, the DDT shows how differences in the input to the S-box translate into differences in the output.

The entire algorithm for the computation of the DDT entries is summarized in Algorithm 2.

The question remains about what defines a secure S-box $\mathcal{S}$. After having computed the DDT, it is possible to analyze the entries in the matrix. Every DDT in our examples has the property of having the value 16 (probability $p = \frac{16}{16} = 1$) in the first row and column. If there is no input difference ΔI , the same plaintext gets encrypted twice, resulting in the random oracle giving the same ciphertext back twice. Checking those two will, of course, result in ΔO being zero as well. This property applies to every DDT, and the value 16 might differ depending on the used S-box. Since this does not help us, we consistently disregard the first row and column when analyzing the DDT.

Definition. The *Maximum Differential Probability* (MDP) is the highest probability that a specific output difference occurs given a specific non-zero input difference for an S-box $\mathcal{S}$. This value is obtained from the Differential Distribution Table (DDT). Formally defined by Christina Boura [Bou18], it is represented as:

$$MDP_f := \max_{\Delta u \neq 0, \Delta v} DP_f(\Delta u \rightarrow \Delta v),$$

Algorithm 2 Calculate Differential Distribution Table (DDT)

Require: S-box $\mathcal{S}$ **Ensure:** Differential Distribution Table (DDT)

```
1: Initialize  $DDT$  with zeros
2: for  $\Delta_x$  from 0 to  $\text{len}(\mathcal{S}) - 1$  do                                ▷ Input difference
3:   for  $\Delta_y$  from 0 to  $\text{len}(\mathcal{S}) - 1$  do                                ▷ Output difference
4:     for  $x$  from 0 to  $\text{len}(\mathcal{S}) - 1$  do                                ▷ Input value
5:        $y \leftarrow x \oplus \Delta_x$ 
6:        $z \leftarrow \mathcal{S}[x]$ 
7:        $z' \leftarrow \mathcal{S}[y]$ 
8:       if  $z \oplus z' = \Delta_y$  then
9:          $DDT[\Delta_x][\Delta_y] \leftarrow DDT[\Delta_x][\Delta_y] + 1$ 
10:      end if
11:    end for
12:  end for
13: end for
14: return  $DDT$ 
```

where $DP_f(\Delta u \rightarrow \Delta v)$ is the *Differential Probability* (DP) of the function f , defined as

$$DP_f(\Delta u \rightarrow \Delta v) := \mathbb{P}[f(x \oplus \Delta u) \oplus f(x) = \Delta v] = |\{x \in \mathbb{F}_2^n \mid f(x \oplus \Delta u) \oplus f(x) = \Delta v\}| \cdot \frac{1}{2^n}.$$

2.2.2 Substitution Permutation Networks (SPN)

A *substitution permutation network* (SPN) is a cipher which takes an input and consists of multiple rounds, where in every round there are three steps, as described in the publication by Heys [Hey02], on which we heavily rely on for this subsection.

In every round, there is a *substitution*, a *permutation*, followed by a *key mixing* step. The input is split into four multiple smaller inputs, depending on the sizes used. Multiple equivalent S-boxes are used for the substitution, which causes *confusion* due to their non-linearity. The output of the S-boxes is then used for permutation. The permutation step rearranges the bit positions according to a predefined pattern, ensuring that bits from different S-boxes are mixed to enhance *diffusion*.

In Figure 2.2 is an example of an SPN later used for differential cryptanalysis attacks. The SPN takes a 16-bit input and splits it into four 4-bit inputs. There are then four identical S-boxes, which take 4-bit inputs.

A good practice here is that the output bits of one S-box should be connected to as many input bits of an S-box in the next round. This causes optimal “diffusion”. Both confusion and diffusion are concepts that date back to Shannon’s principle, first published in 1949, and still relevant to this day [Sha49].

An additional step in the SPN is key mixing. There is (ideally) a unique round key for every round, usually derived from a key schedule. Before the plaintext is sent through the first layer of S-boxes, it is XOR-ed to the first round key. This procedure is repeated for

every round where the intermediate result of the current S-box layer is XOR-ed with the current round key after the permutation has been applied. This again enforces Shannon's principle.

2.2.3 Feistel Networks (FN)

As discussed by Heys, Howard M [Hey02], the *Feistel scheme* is still a popular choice, despite the popularity of AES. For this chapter, we will rely on the publications by Biryukov, Alex *et. al* [BLP15], as well as on the publication by Courtois, Nicolas T [Cou04].

Definition. A *Feistel network* is a symmetric structure used in constructing block ciphers, consisting of multiple rounds where the input is split into two halves. In each round, one half of the data (typically the right half) is processed by a round function that depends on the other half (typically the left half) and a round key. The output of this round function is then XORed with the left half. After this operation, the two halves are swapped. This process is repeated for several rounds, typically achieving confusion and diffusion properties to enhance security. This also means that in every round, only half of the data is subject to the round function and then XORed with the other half and the round key, allowing faster computation than SPNs.

Example. For our Feistel network, the 8-bit plaintext M is split into two 4-bit data blocks: $M = D_0 || D_1$. Each round of encryption starts with a pair $D_j || D_{j+1}$ and produces the output $D_{j+1} || D_{j+2}$ with

$$D_{j+2} = D_j \oplus \mathcal{S}(D_{j+1} \oplus K_j).$$

Each round uses a different round key K_j . After three rounds, the ciphertext is $C = D_3 || D_4$. A visualization is found in Figure 2.1.

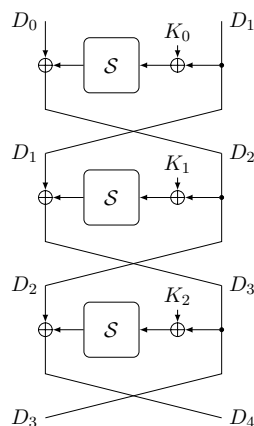


Figure 2.1: Three round Feistel Network

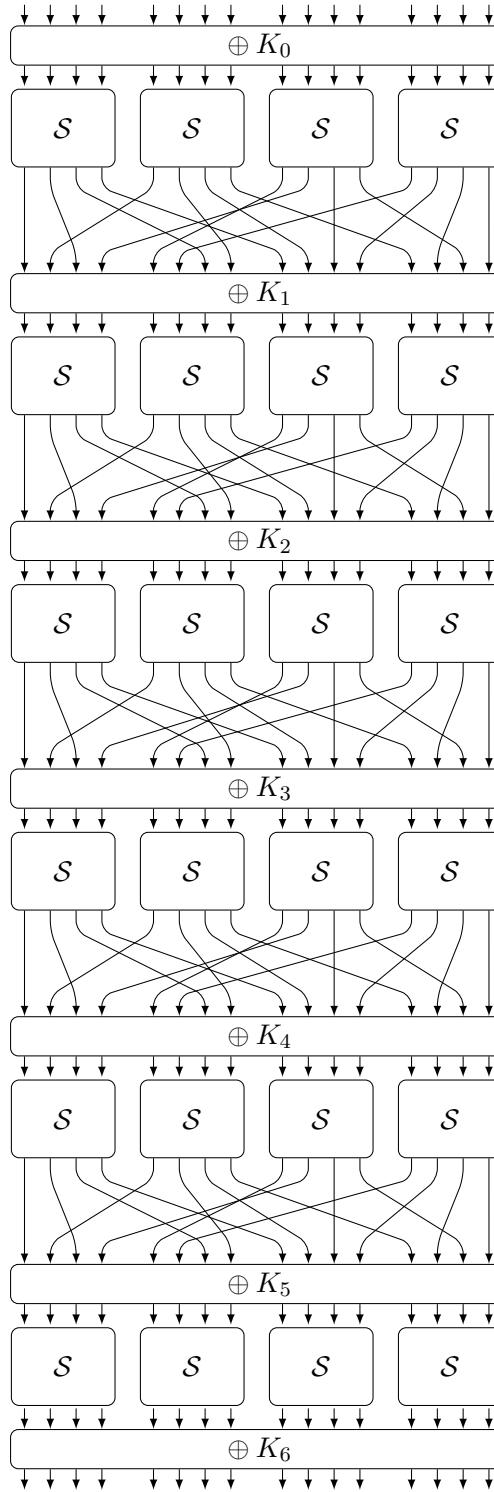


Figure 2.2: Six round Substitution Permutation Network

3 Elliptic Curve Cryptography - Tasks and Implementation

After the terminology explained in section 2.1, we cover the tasks for elliptic curve (EC) cryptography and their implementation details using Python and SageMath.

3.1 Fundamentals: Finite Fields and Binary Fields

The first tasks involve the introduction to *finite fields* and *binary fields*. For these tasks, we refer to the book by Lidl, Rudolf, and Niederreiter, Harald [LN94] unless stated otherwise.

Task. For displaying finite fields and understanding *modular arithmetic*, students receive a small prime p and need to fill out a *Cayley table*. A Cayley table for a group is a mathematical representation that shows the results of applying the group operation (addition or multiplication) to all possible combinations of elements in the field. Each row and column corresponds to an element of the field, and the entries in the table represent the result of combining the corresponding elements using the group operation. In the case of finite fields, the table will have a finite number of rows and columns corresponding to the finite number of elements in the field. To recap modular arithmetic, students must calculate the modular inverses inside the finite field.

For the implementation of this task, there are no real apparent pitfalls. The only thing to consider is to give a small enough prime number such that the order of the finite field is feasible to compute. The calculation process is always the same, and it should just refresh the student's knowledge of modular operations and finding inverses inside a finite cyclic group. Implementation is also relatively straightforward. One iterates in a nested loop for all elements inside the group and performs the group operation. The procedure is summarized in Algorithm 3.

Example. All three tasks for the finite field example $\mathbb{Z}_5$ are visualized below.

$\mathbb{Z}_5$: Cayley tables and inverses:

+	0	1	2	3	4
0	0	1	2	3	4
1	1	2	3	4	0
2	2	3	4	0	1
3	3	4	0	1	2
4	4	0	1	2	3

·	0	1	2	3	4
0	0	0	0	0	0
1	0	1	2	3	4
2	0	2	4	1	3
3	0	3	1	4	2
4	0	4	3	2	1

x	x^{-1}
0	—
1	1
2	3
3	2
4	4

Algorithm 3 Generate Cayley Table for Finite Field

```
1: procedure GENERATECAYLEYTABLE( $p$ )
2:   Let  $G_p$  be the finite field with  $p$  elements.
3:   Initialize an empty  $p \times p$  table  $C$ .
4:   for  $i = 0$  to  $p - 1$  do
5:     for  $j = 0$  to  $p - 1$  do
6:        $C[i][j] \leftarrow (i * j) \bmod p$   $\triangleright$  * represents the group operation
7:     end for
8:   end for
9:   return  $C$ 
10: end procedure
```

Task. For the calculation inside binary fields, we will now give the students a binary field $\mathbb{F}_{2^n}$ with an irreducible polynomial of degree n .

We will give the students two polynomials for the first sub-task, which should be added via XOR calculation.

The second task then involves the multiplication of two polynomials. Here, students might then need to apply the rules for modulo-reduction on the intermediate results from the polynomial multiplication. In some cases, this lets the task be involved, with high degrees within the two polynomials that must be multiplied.

This procedure is also described in Algorithm 4 and the example in paragraph 2.1.1.

Implementation Details. For the tasks regarding the binary fields, we use SageMath. It helps us to find a random irreducible polynomial with the specified degree. The user can also input the two polynomials for the addition and the two for the multiplication task. The input will be a binary string like “101011”, which will then be interpreted as a polynomial with the coefficients set corresponding to the bits. The previous string would be interpreted as $x^5 + x^3 + x + 1$ in this example. Alternatively, the user can also let the tool randomize the polynomials.

In addition, the calculations for addition and multiplication of binary polynomials work pretty well in Python, as the XOR of the coefficients of the same power can be calculated with a regular XOR between the two strings. The procedure for solving the binary field multiplication is summarized in the Algorithm 4.

One pitfall we encounter when creating random polynomials for this task is ensuring the students perform actual computations. The results are trivial if we, for example, get x^0 as the second polynomial. Therefore, we add checks for the generated polynomials to ensure this does not happen.

3.2 Elliptic Curve Groups

The tasks regarding elliptic curve cryptography are split into three sections. The first describes calculating the elliptic curve group points for a given elliptic curve. For this

Algorithm 4 Binary Field Multiplication

```
1: procedure BINARYFIELDMULTIPLY( $a, b, f$ )
2:   Let  $f(x)$  be the irreducible polynomial and  $a, b$  the polynomials to be multiplied
3:   Initialize an empty list  $result$ 
4:   for  $i = 0$  to  $\text{degree}(a) + \text{degree}(b)$  do
5:      $result[i] \leftarrow 0$ 
6:   end for
7:   for  $i = 0$  to  $\text{degree}(a)$  do
8:     for  $j = 0$  to  $\text{degree}(b)$  do
9:        $result[i + j] \leftarrow result[i + j] \oplus (a[i] \cdot b[j])$ 
10:    end for
11:  end for
12:  while  $\text{degree}(result) \geq \text{degree}(f)$  do
13:    Find  $d = \text{degree}(result) - \text{degree}(f)$ 
14:    Calculate  $f(x) \cdot x^d$ 
15:     $result \leftarrow result \oplus (f(x) \cdot x^d)$ 
16:  end while
17:  return  $result$ 
18: end procedure
```

section, we refer to the work of Amara, Moncef, and Siad, Amar [AS11] unless stated otherwise.

Tasks. Students get an elliptic curve over a prime field and must calculate all group points. Afterward, they must draw the curve in a simple coordinate system. This can be done with Algorithm 5 and by applying the Chord and Tangent rule. There are also some extra tasks involving calculating the order of a given point and checking if the given point is a generator of the elliptic curve group.

The user of the task generator can either decide if the tasks should be generated with random calculated values, which aim at finding easy-to-compute tasks, or if they want to adapt or directly give all input values. If they want to enter them manually, the user can first input the wanted prime p , defining the modulus. Also, the user can give the parameters a and b of the Weierstrass equation $y^2 = x^3 + ax + b$ but will be calculated (mod p). However, as discussed in section 3.4, there are potential pitfalls in which the given parameters do not even form an elliptic curve. Since we use SageMath in this task to create and calculate the elliptic curves, we would get an exception error from the SageMath library. Therefore, the user input is first checked. For this check to work, we first construct an elliptic curve with the passed parameters inside a “try & catch” block. If SageMath can create an elliptic curve, it is possible to use the built-in function from SageMath on the created elliptic curve object, which directly gives the order of the elliptic curve. The tool here checks the range of the generated elliptic curve, and if the

elliptic curve has too few or too many group points, other values for a and b are chosen at random until the criteria are met.

The computation of the group points can be summarized in Algorithm 5.

Algorithm 5 Compute Points on Elliptic Curves

```

1: Input: Elliptic curve  $E : y^2 = x^3 + ax + b$  over finite field  $\mathbb{F}_p$ , where  $p$  is a prime.
2: Output: Set of points on  $E$ .
3: Initialize a look-up table (LUT) for  $y^2 \bmod p$  for all numbers inside the field.
4: for  $x$  in  $\mathbb{F}_p$  do
5:   Compute  $y^2 \leftarrow x^3 + ax + b \bmod p$ .
6:   Find  $y$  such that  $y^2 = x^3 + ax + b \bmod p$ .
7:   if  $y$  is not in the LUT then
8:     Continue to the next  $x$ .
9:   end if
10:  Add the points  $(x, y)$ , to the set of points on  $E$ .
11:       $\triangleright$  Usually, there are two points  $(x_1, y), (x_2, y)$  added for every valid  $y$ .
12:       $\triangleright$  This happens since we solve for  $y^2$ .
13:       $\triangleright$  It can be only one point when it holds that  $P = -P$ .
14: end for
```

The Chord and Tangent rules are used for geometric interpretation. Working with these rules and modular arithmetic, it is easy to implement the calculation of elliptic curve groups in regular Python.

One thing to pay attention to is to give a point where it is easy to determine its order. This is especially easy if a *log/antilog table* is given. This table consists of points and their corresponding multiples. These tables usually consist of one generator, called the base point, and the results of $2P, 3P, \dots$, until the neutral element is reached again. For generators, this happens after $|G|$ multiplications. These tables are used for easy computation of scalar multiplication, as seen in the below example.

Example. In this example, the log/antilog table for the elliptic curve $y^2 = x^3 + x$ over $\mathbb{Z}_7$ with base point $P = (3, 3)$ is given in Table 3.1.

k	0	1	2	3	4	5	6	7
$k \cdot \alpha$	$\mathcal{O}$	$(3, 3)$	$(1, 4)$	$(5, 5)$	$(0, 0)$	$(5, 2)$	$(1, 3)$	$(3, 4)$

Table 3.1: Log/Antilog Table

A simple method to now check if, for example, the point $(1, 4)$ is a generator would be to replace it with $2 \cdot (3, 3)$. Then, we can calculate $2 \cdot (1, 4)$ as $2 \cdot 2(3, 3) = 4 \cdot (3, 3) = (0, 0)$. Now we again add $(1, 4)$, meaning $(0, 0) + 2 \cdot (3, 3)$. This results in $(1, 3)$. Adding $(1, 4)$ will result in the final value of $8 \cdot (3, 3)$ which is $\mathcal{O}$. Since $(1, 4) = 2 \cdot (3, 3)$ and we need $8 \cdot (3, 3)$ to reach the neutral element $\mathcal{O}$, we know the order of $(1, 4)$ is only 4. This also means that $(1, 4)$ is not a generator. This is easily seen with the help of Table 3.1 and

without needing actual elliptic curve point calculation. This is a significant speedup compared to the calculation with the Chord and Tangent rule.

3.3 ElGamal encryption for Elliptic Curves

Elliptic curves can be applied to some encryption or key-exchange schemes. Some of these are the elliptic curve Diffie–Hellman key agreement and the elliptic curve ElGamal encryption. As stated by Heys [Hey02], in 1985, T. ElGamal proposed a public key cryptosystem based on the *discrete logarithm problem* (DLP) [Gor05]. By coincidence, there was also a proposal for a cryptosystem relying on elliptic curves in the same year by Neil Koblitz and Viktor Müller. Elliptic curves can be used for ElGamal. This is helpful because the algorithm that makes factoring or recovering the DLP easier does not apply to elliptic curve groups. This also helps reduce key size and parameters needed while maintaining the desired security level. For this section, we will go over the elliptic curve ElGamal encryption and refer to the publication by Sutikno, S. and Surya, A. and Effendi, R. [SSE98] unless stated otherwise.

Task. Students must first compare the regular ElGamal and the elliptic curve ElGamal encryption to understand better how ElGamal works. Afterward, they will need to perform the encryption process themselves.

We give the curve and a base point P , which is a generator, and a random point on the elliptic curve group, which is the message M to be encrypted. Also, a public key Y is given, again a point on the elliptic curve group, including the ephemeral key size and parameters needed k .

Students must compute the pair (r, s) as the private key, like in regular ElGamal encryption. However, the formulas for r and s are different. The adapted formula for calculating the private key is

$$r = k \cdot P \tag{3.1}$$

and

$$s = k \cdot Y + M. \tag{3.2}$$

For this procedure of point calculation, there are techniques like “Square and Multiply” or even simpler methods which involve the given log/antilog table, as described in the example in section 3.2.

After having encrypted the message M , students must decrypt C after finding the private key.

Definition. *Euler’s totient function*, denoted as $\phi(n)$, counts the number of positive integers less than n that are coprime to n , i.e., they share no common factors other than 1.

Implementation Details. In elliptic curve ElGamal, the range of the ephemeral key k requires adjustment compared to regular ElGamal. In both schemes, k is chosen modulo the size (order) of the group generated by the generator. In the finite-field version of the algorithm, this means modulo $\phi(p) = \text{modulo } p - 1$; in other words, in the range 0 through $p - 2$ (where the values 0 and 1 are excluded because they are trivially breakable). In the elliptic curve version, the group size will be approximately but usually not precisely p .

This can (and did) cause some confusion for students in the case of $k \neq p$ when calculating Equation 3.1 and Equation 3.2.

We use SageMath to implement the ElGamal encryption and decryption. The curve parameters can be set similarly as in section 3.2. After this, the generator point P for this task is chosen randomly, and the public key Y , which should be a generator point but might not be one because our curves are not of prime order. Finally, the ephemeral key k is chosen within the previously defined adapted boundaries.

Implementing the log/antilog table or general calculations is done with simple range-based loops. Intuitively, faster algorithms can be used on paper, especially once the log/antilog table is given, as described in the example in section 3.2. However, we stick to the simple approach for these tasks as the loops do not cause significant computation time for such small examples.

It is, however, still noteworthy that these problems are hard to solve for huge primes and elliptic curve groups with a large order. This is due to them being based on the *Discrete Logarithm Problem* (DLP), or in this example on its particular case, the *Elliptic Curve Discrete Logarithm Problem* (ECDLP), as also seen in the publication by Alfred Menezes [Men01].

3.4 Security Aspects of Elliptic Curves

As mentioned in subsection 2.1.3, there are some possibilities of elliptic curve parameters causing security hazards. This chapter covers them and explains how the student tasks are generated, as this task has a significant pitfall.

Tasks. Students get four or more potential candidates for secure elliptic curves. They need to analyze the parameters by calculating the trace of Frobenius and the discriminant. The task generator is currently set to give one example of each: a secure, a singular, a supersingular, and an anomalous curve, but the lecturer can also change this if needed in the generated L^AT_EX file before creating the final PDF. The lecturer can also decide how many curves they want in the final PDF by cherry-picking the generated curves.

Implementation Details. The lecturer can define the prime number q for the modulus $\mathbb{Z}_q$ and the order n defining the number of group points. Then, the tool will generate *anomalous*, *singular*, *supersingular*, and secure curves. From the generated curves, one is randomly selected for each of the four named types. For the trace of Frobenius, we can

easily let SageMath construct the elliptic curve, which gives us the cardinality with the built-in function.

A pitfall we encounter happens at singular curves. A singular curve is not an actual elliptic curve. This causes the SageMath constructor to fail with an exception, and it will not give us the cardinality of the curve. Since we need it for the task and cannot choose a random value as the order, we calculate the order of the singular curve with the *Legendre symbol*.

Definition. The *Legendre symbol* is a mathematical concept used in number theory to determine whether a given number is a quadratic residue modulo a prime number. The computation is summarized in Algorithm 6. The algorithm uses *Euler's criterion* in line 5, a well-known number theory method for computing the Legendre symbol.

Algorithm 6 Compute Legendre Symbol

```

1: function LEGENDRE_SYMBOL( $a, p$ )
2:   if  $a \bmod p = 0$  then
3:     return 0
4:   end if
5:   return  $a^{(p-1)/2} \bmod p$ 
6: end function

```

The Legendre symbol can be used for calculating the cardinality of elliptic curve groups given in the Weierstrass equation by considering its relationship with *quadratic residues* modulo a prime p . In the context of elliptic curves, the cardinality of the group is related to the number of points on the curve over a finite field.

Definition. *Quadratic residues* in the context of elliptic curves are values x for which there exists a corresponding y such that $y^2 \equiv x^3 + ax + b \pmod{p}$, where a and b are coefficients defining the elliptic curve equation and p is a prime number, defining the modulus and the finite field $\mathbb{F}_p$. These residues are essential in determining the existence of points on the curve, contributing to the cardinality calculation of the elliptic curve group over a finite field.

The idea is also based on our previous algorithm for computing points on an elliptic curve, as seen in Algorithm 5. We can use this to calculate the order of the elliptic curves given via the Weierstrass equation $y^2 = x^3 + ax + b$ in Algorithm 7.

With this algorithm, we can circumvent the SageMath exceptions and still calculate the curves' cardinality. These functions also help us to verify the other orders from secure, anomalous, and supersingular curves.

Algorithm 7 Compute Elliptic Curve Order Using Legendre Symbol

```
1: function GET_EC_ORDER_LEGENDRE(prime, a4, a6)
2:   order  $\leftarrow$  1  $\triangleright +1$  for neutral element  $\mathcal{O}$ 
3:   for  $x$  in  $0, 1, \dots, \text{prime} - 1$  do
4:      $y^2 \leftarrow (x^3 + a4 \cdot x + a6) \bmod \text{prime}$ 
5:     ls  $\leftarrow$  legendre_symbol( $y^2$ , prime)
6:     order  $\leftarrow$  order + ls + 1
7:   end for
8:   return order
9: end function
```

Remarks. This concludes our generated tasks for elliptic curve cryptography. Most of the generated tasks rely on SageMath. However, sometimes, we need to manually compute some checks, as seen in section 3.4 in the case of singular curves and their cardinality.

One task not generated by the tool is automatically added to the generated tasks for binary fields but always stays the same. The task is to compute the *Extended Euclidean Algorithm* to find the modular inverse of a given polynomial, as seen below.

Example. Find the inverse of $x^4 + x^3 + x^2 + x + 1$.

Hint: Use the Extended Euclidean Algorithm with polynomials instead of numbers.

4 Differential Cryptanalysis - Tasks and Implementation

In the coming sections, we review the tasks for the fundamentals of the S-box and the DDT and cover three simple examples of differential cryptanalysis applied to different toy block ciphers.

For this chapter, we heavily rely on the publication by Heys, Howard M [Hey02] and the definitions previously defined in chapter 4.

4.1 Differential Distribution Table (DDT)

The differential distribution table is an essential building block for differential cryptanalysis, as also described in subsection 2.2.1.

We first need to understand how an S-box and its corresponding DDT and MDP work before performing differential cryptanalysis on small block ciphers.

Tasks. We are given a S-box and its DDT. In the DDT, some rows or columns are missing. The task is to fill out those to get a final DDT where every element is correctly inserted.

When creating the task, we can set which rows and columns should be missing. Afterward, when we want to solve the task, we must also compute the MDP, as described in subsection 2.2.1.

For this task, there can be either a lot of calculation involved if the students choose the formal approach with the nested loops as seen in Algorithm 2 or they can solve the DDT like a Sudoku.

The DDT has this convenient property where each row and column sum up to 16 in our examples, summing up always to probability $p = 1$. Knowing this, and if the lecturer gives some nice rows and columns where it is easy to see which values one needs to fill in, this task can also become quite fun for students.

Implementation Details. The DDT can be calculated as explained in Algorithm 2. Alternatively, SageMath helps us. A function from the library calculates the DDT directly after we construct the S-box as an object.

For the MDP, the same holds after we obtain the DDT. This also helps us a lot, such that we can get the DDT and MDP from two different sources, verifying that our implementation works.

A problem we encounter is the creation of the S-boxes. As mentioned in subsection 2.2.1, not every LUT is suitable for a non-linear S-box. Coming up with a good S-box by randomly creating a LUT until an S-box and its DDT with the wanted characteristics are made is not an optimal solution.

Therefore, we settle for creating an *affine-equivalent S-box*, based on a technique also seen in the publication by Sakalli *et. al* [SBSB06]. The approach is summarized in Algorithm 8 in which we utilize SageMath and the Python library *numpy* for helper functions for matrix calculations.

Algorithm 8 Generate Affine Equivalent S-Box

Require: S-Box $\mathcal{S}$

```

1: function AFFINE_EQUIVALENT_SBOX( $\mathcal{S}$ )
2:   Generate random invertible 4x4 matrices  $A_1$  and  $A_2$ 
3:   Generate random vectors over GF(2) of size 4  $b_1$  and  $b_2$ 
4:   new_sbox_list  $\leftarrow$  []
5:   for  $i$  in  $0, 1, \dots, 15$  do
6:     Convert  $i$  to vector over GF(2)  $x$ 
7:     new_x  $\leftarrow A_1 \cdot x + b_1$ 
8:     value_sbox  $\leftarrow \mathcal{S}(\text{new\_x})$ 
9:     final_value  $\leftarrow A_2 \cdot \text{value\_sbox} + b_2$ 
10:    new_sbox_list.append(final_value) ▷ Note: This is a binary string
11:   end for
12:   Convert list of vectors new_sbox_list to list of integers new_sbox
13:   return new_sbox
14: end function

```

This discovery also helps us to find suitable S-boxes for the other tasks, which will be discussed in the upcoming subsection.

To check if two S-boxes are equivalent, one could check if the rows and columns of the DDT are still the same, just swapped, and if the MDP (in our case $p = 1$) remains the same.

4.2 Simple 1-Round Toy Cipher Cryptanalysis

After having covered the basics about the S-box and the DDT, it is time for students to check their knowledge in a differential attack on a 1-round toy cipher. For this chapter, we rely on the presentation about differential cryptanalysis by Christina Boura and her example of a 1-round toy cipher [Bou18].

The toy cipher consists of one plaintext message m which the attacker can choose, two distinct keys k_1 and k_2 which are unknown to the attacker, and an S-box $\mathcal{S}$, which is known to the attacker. The attacker can also query the encryption of m to get the

ciphertext c . The intermediate results u and v are used for the differential attack. The toy block cipher is visualized in Figure 4.1.

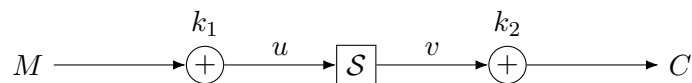


Figure 4.1: A simple cipher diagram

Differential Attack Procedure. The main idea is to find two plaintext messages m_1 and m_2 , which ideally are not identical. Otherwise, we have the problem that $\Delta I = 0$ is of no value for differential cryptanalysis. We encrypt m_1 and get the first ciphertext c_1 . Then we do the same for m_2 to get c_2 . As the attacker, we now know the two plaintext-ciphertext pairs $(m_1, c_1), (m_2, c_2)$. The next step is to compute

$$u_1 \oplus u_2 = (m_1 \oplus k_1) \oplus (m_2 \oplus k_1) = m_1 \oplus m_2.$$

The setup we now describe is visualized in Figure 4.2.

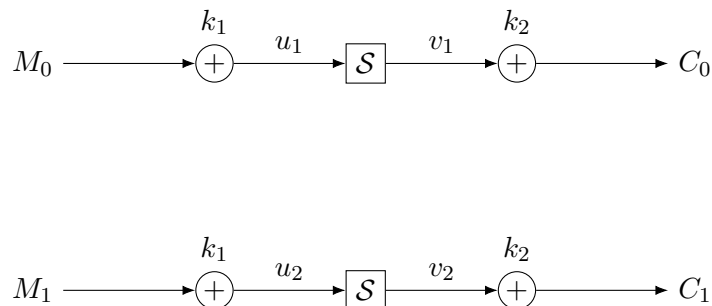


Figure 4.2: The setup for the differential attack

By knowing $u_1 \oplus u_2 = m_1 \oplus m_2$, we need to perform a *meet-in-the-middle-attack* by continuing the attack with the ciphertext difference and trying to match it to $u_1 \oplus u_2$. We know c_1 and c_2 . What we do not know is k_2 . For this, we iterate over all possible k_2 values, which always range from 0 to 15 in our tasks, as we have 4-bit keys. Then, for every k_2 guess, we have $v = c \oplus k_2$.

Having $\mathcal{S}^{-1}$, we continue the attack. We calculate $\mathcal{S}^{-1}(v_1)$ and $\mathcal{S}^{-1}(v_2)$.

For the correct guess t of k_2 it then holds that

$$u_1 \oplus u_2 = \mathcal{S}^{-1}(v_1) \oplus \mathcal{S}^{-1}(v_2) = \mathcal{S}^{-1}(t \oplus c_1) \oplus \mathcal{S}^{-1}(t \oplus c_2).$$

We then always have at least two candidates for k_2 that satisfy this constraint. We can either continue with a reduced key space for k_2 or retry another combination of m_1 and m_2 until we have the minimal number of possible candidates, which is two.

With the obtained possible k_2 candidates we can also derive k_1 by

$$k_1 = m_1 \oplus \mathcal{S}^{-1}(c_1 \oplus k_2).$$

This leaves us either with a final keypair (k_1, k_2) or multiple possible candidates, which means we have successfully performed our attack. If we want to verify them, we can plug in the keys and check if we get the same ciphertexts.

The entire attack can be summarized in Algorithm 9, heavily based on the work from Christina Boura [Bou18].

Algorithm 9 Differential Key Recovery Attack on Toy Cipher

Require: Plaintext-ciphertext pairs (m_1, c_1) and (m_2, c_2)

- 1: Compute $u_1 \oplus u_2 = m_1 \oplus m_2$ ▷ This is our known plaintext difference
 - 2: Compute inverse of S-box $\mathcal{S}^{-1}$
 - 3: Compute the ciphertext difference $v_1 \oplus v_2 = c_1 \oplus c_2$
 - 4: **for** each possible guess t of k_2 from 0 to 15 **do** ▷ Iterate over all 4-bit values
 - 5: Compute $v_1 = c_1 \oplus t$ and $v_2 = c_2 \oplus t$ ▷ Apply the guessed k_2 to the ciphertexts
 - 6: Compute $u'_1 = \mathcal{S}^{-1}(v_1)$ and $u'_2 = \mathcal{S}^{-1}(v_2)$ ▷ Apply the inverse S-box to the intermediate values
 - 7: Compute $u'_1 \oplus u'_2$
 - 8: **if** $u'_1 \oplus u'_2 = u_1 \oplus u_2$ **then** ▷ Check if the difference matches the known plaintext difference
 - 9: Store t as a candidate for k_2 ▷ This guess for k_2 is valid
 - 10: **end if**
 - 11: **end for**
 - 12: **if** more than 2 candidates for k_2 are found **then**
 - 13: Use another plaintext-ciphertext pair to narrow down the candidates
 - 14: **end if**
 - 15: Find k_1 by the equation $k_1 = m_1 \oplus \mathcal{S}^{-1}(c_1 \oplus k_2)$ using the remaining candidates for k_2
 - 16: Verify the key pair (k_1, k_2) by checking if they produce the correct ciphertexts for the given plaintexts
-

Implementation Details. We use an affine equivalent S-box as explained in section 4.1. Inverting the S-box is also very easily done in Python. We develop two random plaintext messages and keys where each has 4 bits. We also check that $m_1 \neq m_2$.

The Python code can also recover all possible key pairs, and we can verify the correctness as we have the correct keys stored, which we use for calculating c_1 and c_2 . The solution inside the generated PDF lists the actual used keys k_1 and k_2 .

4.3 Substitution Permutation Network (SPN) Cryptanalysis

The substitution permutation network (SPN) is a widely used block cipher, which relies on Shannon's principle [Sha49].

For this section, we rely on the previous definitions in subsection 2.2.2 and the publication by Heys [Hey02].

Task. We are given an SPN block cipher as illustrated in Figure 4.3. The SPN consists of 6 rounds. Each round comprises three layers: an XOR with a round key, a layer of 4 identical 4×4 -bit S-boxes $\mathcal{S}$, and a bit permutation. In the last round, there is no permutation after the S-box output. We also get two plaintext/ciphertext pairs (M_0, C_0) and (M_1, C_1) .

The task is to show that it is possible to reduce the key space of the last round key $K_{6,i}$ by 2 bits, with i being the index of the *target* S-box, starting with 1 for the leftmost S-box. They also get the hint to analyze how plaintext differences propagate forward and to get some information about the input difference of the target S-box in the last round, the S-box, where the attack can be performed. In Figure 4.3, the target S-box is the rightmost S-box in the last round, as also seen in the complete solution of the task in Figure 4.7. Students receive the hint and also a color highlighting for the target S-box.

Differential Attack Procedure and Example. To explain this attack, we simultaneously solve it for a concrete example, allowing for a more straightforward demonstration of the involved steps. We start a differential attack on the SPN by splitting the 16-bit messages into four 4-bit nibbles. We obtain four 4-bit nibbles for M_0 and four 4-bit nibbles for M_1 .

In our example, the two plaintext/ciphertext pairs (M_0, C_0) and (M_1, C_1) were generated with the same key:

$$\begin{aligned} M_0 &= 0100\ 0110\ 1010\ 1001 & \rightarrow & C_0 = 1110\ 0101\ 0011\ 0111 \\ M_1 &= 1000\ 1010\ 0110\ 0101 & \rightarrow & C_1 = 1101\ 1001\ 1001\ 0101 \end{aligned}$$

We compute $\Delta M = M_0 \oplus M_1$ and $\Delta C = C_0 \oplus C_1$. In our example $\Delta M = \text{c c c c}$ and $\Delta C = 3 \text{ c a } 2$. The tasks are always constructed in a way that the input difference ΔM has the same 4-bit nibbles, meaning $M_{0,i} \oplus M_{1,i}$ is the same $\forall i \in \{0, 3\}$. More precisely, this XOR difference then always results in the value that allows us to guarantee a specific differential transition, a so-called *linear structure*. For this, we need to have an S-box $\mathcal{S}$ and its corresponding DDT, where the MDP = 1, as seen in Table 4.1 and Table 4.2.

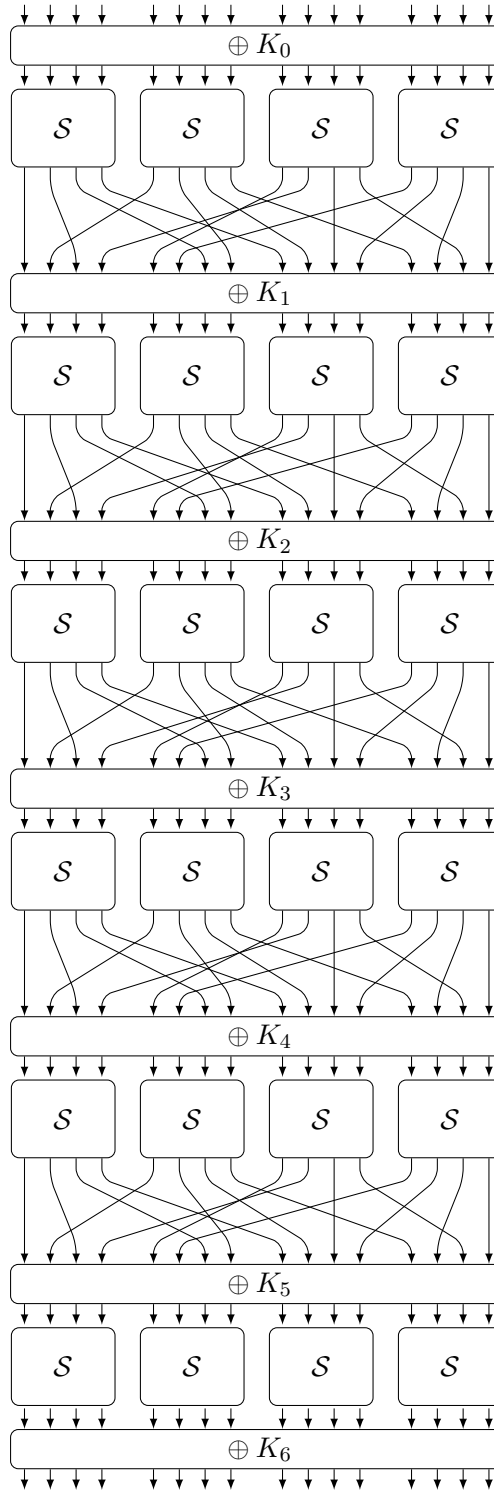


Figure 4.3: Six round Substitution Permutation Network

Note the linear structure $c \rightarrow 8$ highlighted in Table 4.2, meaning that for input difference c , we always get the output difference 8 and vice-versa.

Table 4.1: S-box $\mathcal{S}$

x	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
$\mathcal{S}(x)$	2	0	4	3	9	5	6	7	1	d	e	f	a	8	c	b

Table 4.2: Differential distribution table of the S-box $\mathcal{S}$ with highlighted linear structure

I \ O	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	16	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
1	-	4	4	-	-	-	-	4	-	-	-	-	4	-	-	-
2	-	-	4	4	-	-	4	-	-	-	-	-	-	-	-	4
3	-	4	-	4	4	-	-	-	-	-	-	-	-	-	4	-
4	-	-	4	-	4	4	-	-	-	-	-	4	-	-	-	-
5	-	-	-	4	-	4	-	4	-	4	-	-	-	-	-	-
6	-	-	-	-	4	-	4	4	-	-	-	-	-	4	-	-
7	-	4	-	-	-	4	4	-	-	-	4	-	-	-	-	-
8	-	-	-	4	-	-	-	-	-	-	4	-	4	4	-	-
9	-	4	-	-	-	-	-	-	-	-	-	4	-	4	-	4
a	-	-	-	-	-	4	-	-	-	-	-	-	4	-	4	4
b	-	-	4	-	-	-	-	-	-	4	-	-	-	4	4	-
c	-	-	-	-	-	-	-	-	16	-	-	-	-	-	-	-
d	-	-	-	-	4	-	-	-	-	4	4	-	-	-	-	4
e	-	-	-	-	-	-	-	4	-	-	4	4	-	-	4	-
f	-	-	-	-	-	-	4	-	-	4	-	4	4	-	-	-

In Figure 4.4, we see that each of the four S-boxes has precisely two bits set for the input difference c and one output bit set after the S-box for the difference 8. The arrows at the top of the figure represent the input differences in binary representation, and the arrows after the S-box represent the output difference. The arrows and the S-box are blue because we know the exact differences.

We can propagate the output bits of the active S-boxes and see which input bits they connect to for the S-boxes of the next round. In the case of this S-box or one of its affine equivalent S-boxes, we can have the characteristic that we can guarantee an output and input difference for one specific combination. The S-box and DDT in the examples always have a differential transition where the input difference has two bits set, whereas the output difference only has one bit set. In the example S-box, this is fulfilled, as c has the binary representation 1100 and 8 has the binary representation 1000. This can also be seen in Figure 4.4. Due to the linear structure, we know the output difference of the first round 8 8 8 8. Note that only 4 bits are set.

Those bits connect to two S-boxes in the second round, resulting in those two S-boxes having the same input difference as in the first round, while the two others do not have any known input difference. In Figure 4.4, this can be seen in the second round, where the first two S-boxes remain blue, and the other two are no longer colored since we have no *active* bits in them, which lets us know the output difference.

It can be observed in Figure 4.4 that in the first round, we have precisely two active input bits for each of the four S-boxes and then precisely one active output bit for each S-box. The active output bits connect to precisely two S-boxes in the second round, and they have the same input difference as in the first round. The output bits are then again the same, as the input difference remains unchanged. However, this time, only two S-boxes are active, so we only have two output difference bits that are active.

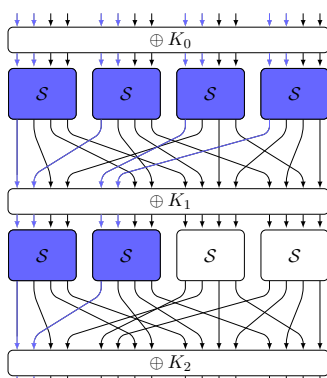


Figure 4.4: First two rounds of the Substitution Permutation Network Solution

The permutations now lead these two active bits to one S-box in the third round. The input difference is now the same again as in the previous S-boxes. This means we have one active output bit, which then connects to one S-box in the fourth round. In Figure 4.5, this can be observed with the blue leftmost S-box in the third round.

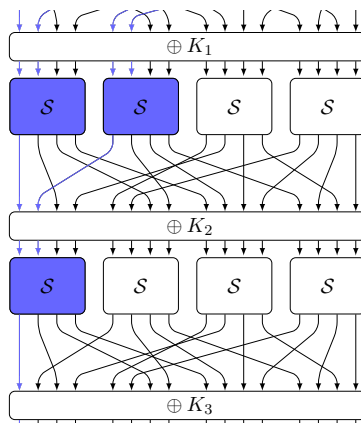


Figure 4.5: Second and third round of the Substitution Permutation Network Solution

In the fourth round, we only have a single active bit for the input difference in a single S-box. Therefore, guaranteeing the linear structure is no longer possible, which helps predict the output difference bits with certainty. In the example, the only active input bit for the S-box in the first round results in input difference 8, as seen in Figure 4.7. In the DDT in Table 4.2, it can be seen in the row for 8, there are the possible output differences $\{3, a, c, d\}$, which in binary representation is $\{0011, 1010, 1100, 1101\}$. In none of these four nibbles, one bit is definitely set or not set in all four of them. Therefore, we must assume that all output difference bits are potentially active. As seen in Figure 4.6, we color the S-box in the fourth round red and all four output difference bits, assuming all of them to be potentially active.

This leads to three S-boxes being active with an unknown input and output difference in the fifth round, as seen in Figure 4.6. We propagate all output bits of each of the three S-boxes. We see that the rightmost S-box in the last round has two unknown input difference bits and two input difference bits, which come from the one S-box from the previous round, which was definitely inactive.

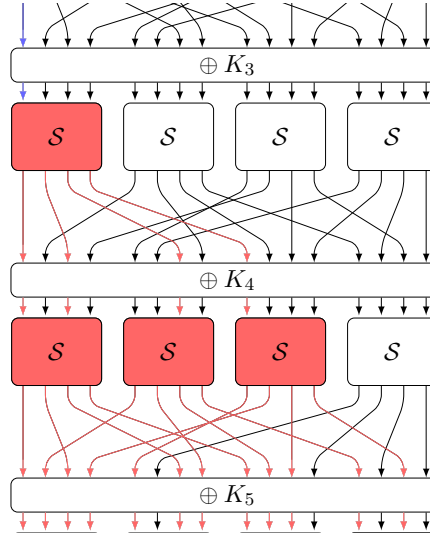


Figure 4.6: Fourth and fifth round of the Substitution Permutation Network Solution

There is no permutation layer in the last round, as seen in Figure 4.3. This means that the output of the S-box is then directly XOR-ed with the last round key and left as the final resulting ciphertext.

A complete visualization of the propagation of the input differences until the sixth round is included in Figure 4.7.

We compute the differences backward, starting with the known output difference ΔC . Since the target S-box is the rightmost S-box in the sixth round, we denote $\Delta S_{6,4}^{out} = 0010$.

Important to note is that the round key addition does not influence the differential attack, for the same reason as in section 4.2. The only round key we need to consider is K_6 in the last round, which we want to recover partially with the differential attack.

We can reduce the active input difference bits by propagating the differences from the top. In Figure 4.6, we can see in the last round that only two red arrows are found for the input difference of the target S-box. This helps to reduce the potential input difference bits to $\Delta S_{6,4}^{in} = x0x0$. This means that the input difference bits are never set before the last S-box for 2^0 and 2^2 . This means that the only possible values would be $\{0x0, 0x2, 0x8, 0xa\}$ and since $\Delta C_4 = 2$, the difference 2 is correctly in the set of the possible candidates.

Since the output difference of the last S-box equals the nibble, it is possible to check column 2 in the DDT in Table 4.2. This gives $\Delta S_{6,4}^{in} = \{0001, 0010, 0100, 1011\}$. We can then compute the exact value of $\Delta S_{6,4}^{in}$. It holds that

$$\Delta S_{6,4}^{out} = 0010 \implies \Delta S_{6,4}^{in} = \{0001, 0010, 0100, 1011\} \implies \Delta S_{6,4}^{in} = 0010.$$

The target S-box is the rightmost S-box in this example. Therefore, we need the last four nibbles $C_{0,4}$ and $C_{1,4}$ of the ciphertexts C_0 and C_1 , which are 7 and 5 in decimal.

We can recover the exact value of $S_{6,4}^{out}$ with those values. For this, we need the inverse S-box $\mathcal{S}^{-1}$ of Table 4.1 and iterate over all possible values of K_6 . The procedure is visualized in Table 4.3, whereas some rows were left out due to simplicity. We only care about the rows where $\Delta \mathcal{S}^{-1} = \Delta C_4$, which is 2 in our example.

Table 4.3: Recovering $S_{6,4}^{out}$

$C_{0,4} \oplus K_{6,4}$	$C_{1,4} \oplus K_{6,4}$	$\mathcal{S}^{-1}(C_{0,4} \oplus K_{6,4})$	$\mathcal{S}^{-1}(C_{1,4} \oplus K_{6,4})$	$\Delta \mathcal{S}^{-1}$
0x0	0x2	0x1	0x0	0x1
0x1	0x3	0x8	0x3	0xb
0x4	0x6	0x2	0x6	0x4
0x5	0x7	0x5	0x7	0x2
0x8	0xa	0xd	0xc	0x1
0x9	0xb	0x4	0xf	0xb
0xc	0xe	0xe	0xa	0x2
0xd	0xf	0x9	0xb	0x2

With this information, we can calculate the remaining values needed for reducing the possible keys K_6 for the last 4 bits. From Table 4.3 we know $S_{6,4}^{out} = \{5, 7, d, f\}$. It then holds that

$$K_{6,4} = S_{6,4}^{out} \oplus C_{0,4} = \{0101, 0111, 1101, 1111\} \oplus 0111 = \{0010, 0000, 1010, 1000\}.$$

This completes the attack and leaves 0, 2, 8, a as candidates for the key $K_{6,4}$. With four possible remaining values, we have a key space of 2^2 instead of one with 2^4 , meaning the key space has successfully been reduced by 2 bits.

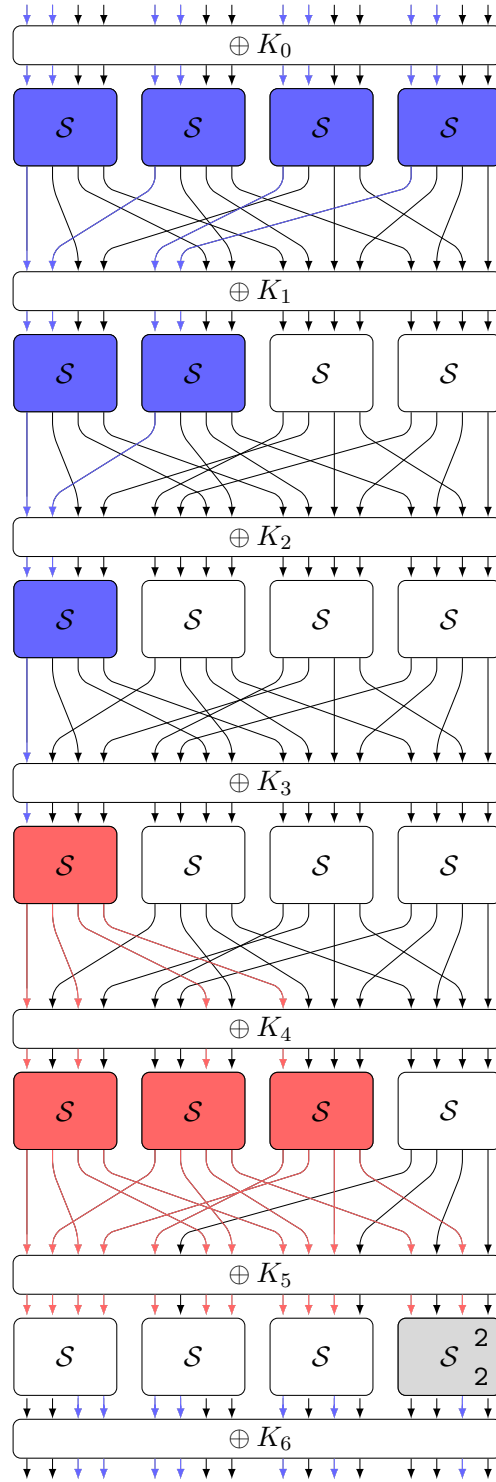


Figure 4.7: Six round Substitution Permutation Network Full Solution

Implementation Details. To create this task, we rely on affine equivalent S-boxes. Most S-boxes might not fulfill the requirement of the guaranteed linear structure due to the corresponding DDT not having an MDP of 1 and the transition from two input difference bits to one output difference bit. So, the generator for this task runs in a loop and tries possible combinations until it finally finds a working S-box and a combination of plaintext/ciphertext pairs. This is still reasonably fast and takes only a few seconds, especially if the S-box from Table 4.1 or one of its affine equivalent S-boxes is used.

Having a suitable S-box and its DDT, we can take the row index as the `target_difference`. With this value taken, we compute the 16-bit plaintext pair for M_0, M_1 . We only need a constraint for the pair, given in Equation 4.1.

$$M_1 = M_0 \oplus \text{target_difference} \quad (4.1)$$

With the known values for M_0 and M_1 , it is possible to calculate the input difference $\Delta I = M_0 \oplus M_1$. The round key additions do not change anything regarding the input difference bits. This is why we can directly map the input difference bits to the S-boxes.

In the first round, obtaining the output differences of all four S-boxes is easy, as the setup guarantees a linear structure. This means that every S-box has the same 2-bit input difference, and each S-box produces the same output bit difference. We can obtain the output bit difference for the current round and create a mapping for the input bit difference for the next round. In every round, we store them in a new 16-bit LUT, which bits are active before and after the S-box. Important to note is that the values of the bits need to be handled carefully. In an array, the first element has index 0, but it would represent the input bit difference for 2^3 in the leftmost S-box.

With the index representation in mind, storing the array and checking how the linear structure can be guaranteed is now possible. For this, the permutations need to be set. The permutation should be the same in every round and should have similar traits, such as a nonlinear function similar to an LUT or an S-box.

It is needed to ensure permutations which lead the four single output difference bits in the first round to produce the same 2-bit input difference bit on two S-boxes in the second round and then also have the same 2-bit input difference in the third round in a single S-box. This is only possible when the active S-box in the third round has the same index as one of the two active S-boxes in the second round.

We use *z3* to establish these constraints and get a correct function representing the permutations. More precisely, we construct ourselves an instance of the *z3* constraint satisfaction problem solver and define a function. This function then represents the permutation layer in every round. We also use the *random* library from Python to ensure some randomness in the created examples.

The unique mapping done via the permutation layer should resemble a non-linear LUT, so all values should remain in the range from 0 to `f`.

We define a function $f : \{0, 1, 2, \dots, 15\} \rightarrow \{0, 1, 2, \dots, 15\}$. The function f maps each element i from the set $\{0, 1, 2, \dots, 15\}$ to an integer within the same set.

$$\forall i \in \{0, 1, 2, \dots, 15\}, 0 \leq f(i) \leq 15$$

The function f must be bijective, meaning each value in the set $\{0, 1, 2, \dots, 15\}$ is mapped to a unique value in the set $\{0, 1, 2, \dots, 15\}$.

$$\forall i, j \in \{0, 1, 2, \dots, 15\}, i \neq j \implies f(i) \neq f(j)$$

The permutations of the SPN need to have also the property of causing diffusion, as seen in Shannon's principle [Sha49]. Therefore, we add the constraint that every S-box needs to be mapped to precisely three other S-boxes. This also helps us guarantee the wanted linear structure and having three S-boxes potentially active in the fifth round, as seen in Figure 4.6. The constraint can be written mathematically as

$$\forall i \in \{0, 1, 2, 3\}, \left| \left\{ \left\lfloor \frac{f(4i + k)}{4} \right\rfloor \mid k \in \{0, 1, 2, 3\} \right\} \right| = 3.$$

To ensure the desired diffusion properties of the SPN, we impose specific constraints on the input difference bits in the second round. We begin by fixing two random S-box indices in the second round, which will receive the desired input difference bits. From these two indices, we randomly select one, designating it as the index of the only active S-box in the third round. It is crucial to ensure these constraints adhere to the representation in the LUT and the actual bit values in each S-box.

Given the parameters:

- `output_diff_bit`: the index of the output difference bit.
- `input_diff_bit1` and `input_diff_bit2`: the indices of the input difference bits.
- `fixed_indices_round2`: the fixed indices for the S-boxes in the second round.

The constraints are as follows:

First, we determine the indices for the output difference bit:

$$\text{constraint_indices_f} = \{4i + \text{output_diff_bit} \mid i \in \{0, 1, 2, 3\}\}$$

We determine the indices for the input difference bits:

$$\text{constraint_indices_input} = \{i \cdot 4 + \text{input_diff_bit1}, i \cdot 4 + \text{input_diff_bit2} \mid i \in \text{fixed_indices_round2}\}$$

We use the indices in the function to ensure the function f maps the output difference indices to the input difference indices:

$$f(4i + \text{output_diff_bit}) = \text{constraint_indices_input} \quad \forall i \in \{0, 1, 2, \dots, |\text{constraint_indices_f}| - 1\}$$

These constraints ensure that the function f correctly maps the indices in a way that maintains the required diffusion properties through the second and third rounds of the SPN.

For the third round, a very similar approach is used.

Given the parameters:

- `fixed_index_round3`: the fixed index for the S-box in the third round.
- `input_diff_bit1` and `input_diff_bit2`: the indices of the input difference bits.
- `fixed_indices_round2`: the fixed indices for the S-boxes in the second round.
- `output_diff_bit`: the index of the output difference bit.

The constraints are as follows:

First, we determine the indices for the input difference bits in the third round:

$$\begin{aligned}\text{constraint_round3_input1} &= \text{fixed_index_round3} \times 4 + \text{input_diff_bit1} \\ \text{constraint_round3_input2} &= \text{fixed_index_round3} \times 4 + \text{input_diff_bit2}\end{aligned}$$

Next, we determine the active output bits after the second round using the fixed indices for the S-boxes in the second round:

$$\begin{aligned}\text{active_bits_after_second_round1} &= 4 \times \text{fixed_indices_round2}[0] + \text{output_diff_bit} \\ \text{active_bits_after_second_round2} &= 4 \times \text{fixed_indices_round2}[1] + \text{output_diff_bit}\end{aligned}$$

We then ensure that the function f maps the active output bits after the second round to the input difference bits for the third round. The constraints can be expressed as:

$$\begin{aligned}(f(\text{active_bits_after_second_round1}) &= \text{constraint_round3_input1} \wedge \\ f(\text{active_bits_after_second_round2}) &= \text{constraint_round3_input2}) \\ \vee \\ (f(\text{active_bits_after_second_round1}) &= \text{constraint_round3_input2} \wedge \\ f(\text{active_bits_after_second_round2}) &= \text{constraint_round3_input1})\end{aligned}$$

These constraints ensure that the active output bits after the second round map to the correct input difference bits for the third round, maintaining the desired structure and properties of the substitution-permutation network.

With these constraints set, getting the known output difference after the third round is also possible. We need access to the `z3` function with the constraints we previously set to calculate the known input difference in the fourth round.

It can also happen that the solver returns an unsatisfiable solution. If this happens, we start again until we have a valid permutation layer. We store the result in a regular Python list to work with the permutation layer without requiring `z3`.

With the `z3` function as a Python array, it is easy to calculate the input difference bit for the fourth round. Since we restrict our DDT to not leak any information about a particular bit being indeed active, we want to have a check for undisturbed bits. This means that the single input bit difference caused by the linear structure will not give away any information, leaving the remaining output difference bits in the fourth round all unknown.

The DDT in the examples, as long as affine-equivalent S-boxes of the S-box in Table 4.1 are used, will have four possible output differences, each with a 25% probability. From

those four values, we want that there is not one bit set in all of them and not a bit that is not set in all. This is checked in the DDT when the linear structure is given. In Table 4.2, the linear structure is given by $c \rightarrow 8$, and the input difference 8 does not leak any output difference bits.

Since it is unknown which output difference bits are set in the only active S-box in the fourth round, we assume all of them to be active and use the permutation layer function to see which S-boxes are active in the fifth round.

The rest, again, propagates the possible input and output differences. To check if the pattern is reachable, we also try all 16 possible output differences after the fourth round and check if, in the fifth round correctly, there are only three S-boxes that are active with an unknown input difference. Also, we check if, in the last round, there is one S-box that gets two unknown active input difference bits, and two bits are known to be inactive.

If this constraint is satisfied, it is then possible to randomly select the target 4-bit nibbles of the two 16-bit ciphertexts C_0 and C_1 . We choose random values for the remaining 12 bits, as we only care about the nibbles of the target S-box.

The key recovery process is the same as in Table 4.3. Some checks are added to ensure a final key space of exactly four values for the last round key of the target S-box.

This concludes the task creation and guarantees the creation of an S-box, its DDT, and an SPN, where there is a linear structure as described in this subsection.

Remarks. This task only works with S-boxes that have a MDP of 1 and the differential transition where the input difference bits need to have precisely two bits set, so a value of 0x3 (binary: 0011), 0x5 (binary: 0101), 0x6 (binary: 0110), 0x9 (binary: 1001), 0xa (binary: 1010), 0xc (binary: 1100) and an output difference bit with precisely one bit set, so one of the following values 0x1 (binary: 0001), 0x2 (binary: 0010), 0x4 (binary: 0100), 0x8 (binary: 1000). It is essential since we otherwise cannot traverse the output difference bits in the first few rounds.

The code itself relies on randomly finding good values. Still, due to some constraints and constant checking and retrying, it is already reasonably optimized and can create new tasks within seconds.

z3 itself is deterministic and always returns the first result found. Therefore, we shuffle the values for the S-box indices, the possible values for the ciphertext, and the constraints for the permutation layer randomly, which allows us to get some new random tasks.

4.4 Feistel Network (FN) Cryptanalysis

For this chapter, we rely on the publications by Biryukov, Alex *et. al* [BLP15], as well as on the publication by Courtois, Nicolas T [Cou04].

Task. We have a 3-round Feistel network with three round keys K_0, K_1, K_2 , unknown to the attacker, as visualized in Figure 4.8. We get two 8-bit plaintext-ciphertext pairs (M_0, C_0) and (M_1, C_1) . The task is to reduce the key space of the keys K_0 and K_2 , meaning

the first and last round-key. The given plaintext-ciphertext pairs are only compatible with a single *characteristic*. This allows us to perform a differential cryptanalysis attack where it is possible to directly know the difference of ΔD_2 by computing it from multiple sources.

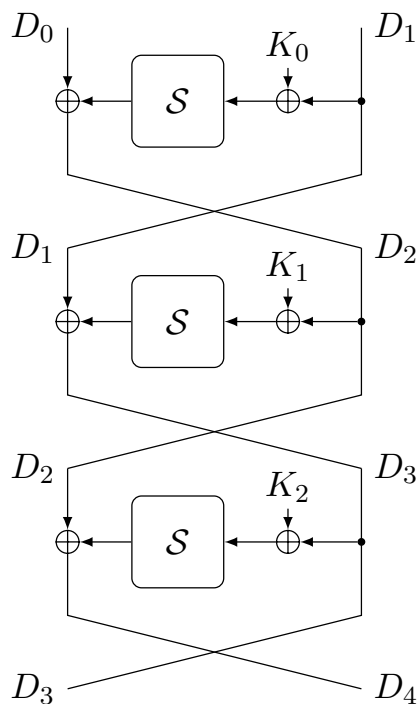


Figure 4.8: Three round Feistel Network

Differential Attack Procedure and Example. To explain this attack, we simultaneously solve it for a concrete example, which allows for an easier demonstration of the involved steps.

In this example the two plaintext/ciphertext pairs (M_i, C_i) were generated with the same key:

$$\begin{aligned} M_0 &= 0110\ 1011 & \rightarrow & C_0 = 1011\ 0001 \\ M_1 &= 1111\ 0010 & \rightarrow & C_1 = 0101\ 1001 \end{aligned}$$

The used S-box and DDT are given in Table 4.4.

We start the attack by calculating $\Delta D_0, \Delta D_1, \Delta D_3, \Delta D_4$. We need to XOR the first 4 bits of M_0 with the first 4 bits of M_1 . This is then the value for ΔD_0 . The same procedure with the last 4 bits gives us ΔD_1 . Doing the same with the two ciphertexts gives us ΔD_3 and ΔD_4 , leaving only ΔD_2 as an unknown value. In our example the values calculated are $\Delta D_0 = 9, \Delta D_1 = 9, \Delta D_3 = e, \Delta D_4 = 8$.

Table 4.4: Definition and differential distribution table (DDT) of the 4-bit S-box $\mathcal{S}$.

x	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
$\mathcal{S}(x)$	2	0	4	3	9	5	6	7	1	d	e	f	a	8	c	b

I\O	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	16	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
1	-	4	4	-	-	-	-	4	-	-	-	-	4	-	-	-
2	-	-	4	4	-	-	4	-	-	-	-	-	-	-	-	4
3	-	4	-	4	4	-	-	-	-	-	-	-	-	-	4	-
4	-	-	4	-	4	4	-	-	-	-	-	4	-	-	-	-
5	-	-	-	4	-	4	-	4	-	4	-	-	-	-	-	-
6	-	-	-	-	4	-	4	4	-	-	-	-	-	4	-	-
7	-	4	-	-	-	4	4	-	-	-	4	-	-	-	-	-
8	-	-	-	4	-	-	-	-	-	-	4	-	4	4	-	-
9	-	4	-	-	-	-	-	-	-	-	-	4	-	4	-	4
a	-	-	-	-	-	4	-	-	-	-	-	-	4	-	4	4
b	-	-	4	-	-	-	-	-	-	4	-	-	-	4	4	-
c	-	-	-	-	-	-	-	-	16	-	-	-	-	-	-	-
d	-	-	-	-	4	-	-	-	-	4	4	-	-	-	-	4
e	-	-	-	-	-	-	-	4	-	-	4	4	-	-	4	-
f	-	-	-	-	-	-	4	-	-	4	-	4	4	-	-	-

To solve ΔD_2 with certainty, we must check the possible differences from multiple sources. We can compute ΔD_2 with certainty because of the characteristic mentioned previously. This is possible because we have multiple sets of potential candidates when calculating ΔD_2 from different directions. Still, in our examples, just one candidate will be present in all sets, as will be later seen in this example. Important to note is that the order of how one wants to calculate all the candidates for ΔD_2 is irrelevant.

By looking at the second round of the cipher in Figure 4.8, it is clear that

$$\Delta D_2 \xrightarrow{\mathcal{S}} \Delta D_1 \oplus \Delta D_3. \quad (4.2)$$

This tells us that the output difference of the DDT is exactly $\Delta D_1 \oplus \Delta D_3$ for S-box input $\Delta D_2 \oplus K_1$. Therefore, with the DDT, we can check the $(\Delta D_1 \oplus \Delta D_3)^{th}$ column, and we get the possible input differences. In our example, we calculate $\Delta D_1 \oplus \Delta D_3 = 9 \oplus e = 7$. We store all nonzero values from the 7th column in a set $W_1 = \{1, 5, 6, e\}$.

In the last round of the cipher in Figure 4.8, it can be observed that

$$\Delta D_3 \xrightarrow{\mathcal{S}} \Delta D_4 \oplus \Delta D_2. \quad (4.3)$$

The DDT gives the possible output differences for input difference ΔD_3 . The nonzero values of the $(\Delta D_3)^{th}$ row of the DDT give a set, but we also need to XOR it with ΔD_4 . This is the second set of possible differences for ΔD_2 , which we store in the set W_2 . In our example, the nonzero values of the DDT at the respective row are $\{7, \mathbf{a}, \mathbf{b}, \mathbf{e}\}$, XOR-ed with ΔD_4 , which is 8, gives us $W_2 = \{\mathbf{f}, 2, 3, 6\}$. We already notice that 6 is the only value that occurs in both W_1 and W_2 , so most likely $\Delta D_2 = 6$ in this example.

The first round of the cipher in Figure 4.8 tells us that

$$\Delta D_1 \xrightarrow{\mathcal{S}} \Delta D_0 \oplus \Delta D_2. \quad (4.4)$$

Since we work with differences, adding K_0 can be neglected. The nonzero values of the DDT at the ΔD_1^{th} row are now taken and XOR-ed with ΔD_0 , which gives us W_3 . In our example the $(\Delta D_1)^{th}$ row of the DDT has the nonzero values $\{1, \mathbf{b}, \mathbf{d}, \mathbf{f}\}$. XOR-ing ΔD_0 to the values gives us $W_3 = \{8, 2, 4, 6\}$. This proves the claim that $\Delta D_2 = 6$.

Important to note here again is that the order of how one calculates W_1, W_2, W_3 does not matter as we only check for the intersecting values, which in our examples are always guaranteed to have only a single difference possible for ΔD_2 after calculating it from multiple sources as explained.

With ΔD_0 and ΔD_2 known, it is possible to find the difference after the first S-box from Equation 4.4. The difference after the first S-box is then equal to $\Delta D_0 \oplus \Delta D_2$, which in our example is $9 \oplus 6 = \mathbf{f}$, stored in ΔO_1 .

With the known output difference of the S-box in the first round, it is possible to reduce the key space of K_0 . For this, we take the last 4 bits of M_0 and M_1 and store them as M'_0, M'_1 . We try for all possible K_0 values (from 0 to 15) to see if they satisfy

$$\mathcal{S}(M'_0 \oplus K'_0) \oplus \mathcal{S}(M'_1 \oplus K'_0) = \Delta O_1.$$

In our examples, we will find precisely four values when trying for K'_0 and, these will be the final candidates for K_0 .

Table 4.5: Recovering K_0

K'_0	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
$\mathcal{S}(M'_0 \oplus K'_0) \oplus \mathcal{S}(M'_1 \oplus K'_0)$	b	d	f	1	d	b	1	f	d	b	1	f	b	d	f	1

In Table 4.6 we can see that the final possible candidates for K_0 are $\{2, 7, \mathbf{b}, \mathbf{e}\}$.

With ΔD_2 and ΔD_4 known, it is possible to find the exact output difference for the S-box in the last round from Equation 4.3. The difference after the last S-box equals $\Delta D_4 \oplus \Delta D_2$, which in our example is $8 \oplus 6 = \mathbf{e}$, stored in ΔO_3 .

With the known output difference of the S-box in the last round, it is possible to reduce the key space of K_2 . For this, we take the first 4 bits of C_0 and C_1 and store them as C'_0, C'_1 . We try for all possible K_2 values (from 0 to 15) to see if they satisfy

$$\mathcal{S}(C'_0 \oplus K'_2) \oplus \mathcal{S}(C'_1 \oplus K'_2) = \Delta O_3.$$

In our examples, we will find exactly four values when trying for K'_2 , and these will be the final candidates for K_2 .

Table 4.6: Recovering K_2

K'_2	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
$\mathcal{S}(C'_0 \oplus K'_2) \oplus \mathcal{S}(C'_1 \oplus K'_2)$	a	7	a	7	b	e	b	e	b	e	b	e	a	7	a	7

In Table 4.6 we can see that the final possible candidates for K_2 are $\{5, 7, 9, b\}$. This completes the attack and the example.

Implementation Details. In this task, the two plaintext messages are chosen randomly and are checked not to have $\Delta M = 0$.

We then let the Feistel encryption be done with three randomly chosen keys. Then, the Python code performs the previously described attack. If it succeeds and the tuples allow precisely only one characteristic compatible, it should also directly reduce the possible keys to 4 instead of 16.

The implementation runs until it finds the suitable values and stops once it has found a random combination of input plaintexts. Alternatively, it would also be specified to specify the wanted input difference ΔI . Still, it would not change much, except for a longer runtime, until it has found two 4-bit plaintexts that satisfy the characteristic. The current runtime also only takes a few seconds to find a new suitable combination of values to ensure the characteristic.

The S-Box is computed with the affine equivalent S-Box algorithm, as described in section 4.1.

The plaintexts get encrypted with the three randomly chosen 4-bit keys K_0, K_1, K_2 . The solution PDF also displays the used keys and the recovered reduced key space for K_0 and K_2 . This also helps to double-check if the produced solution is valid.

Remarks. This concludes our generated tasks for differential cryptanalysis. The tasks cover everything needed to understand this topic. We start with the fundamentals, covering the S-box, the DDT, and the MDP.

Afterward, the first attack on the simple one-round toy cipher represents a warmup attack.

The attacks on the SPN and FN are a good introduction to applying differential cryptanalytical attacks on small ciphers, which are also used in real life but with more rounds and a larger key space.

5 Conclusion

This thesis aims to provide an easily expendable tool with which lecturers can create new tasks for the practicals of the “Cryptography” course [IAI24b]. The new terminal user interface relies on `Click` and no longer has a GUI as in the previous task generator by Romana Gruber [Gru20].

The tool now successfully calculates tasks regarding *elliptic curve cryptography* and *differential cryptanalysis*. The tasks are also generated within seconds and have a reasonably stable runtime and memory complexity if the corresponding parameters are reasonably sized. For example, finding all kinds of singular elliptic curves for a massive prime can take a few minutes, even when using *SageMath*.

The main task of the generator is to design the tool to perform the cryptanalysis of the SPN cipher and come up with constraints that allow for an easy recovery for students. This task is the most involved and needs support from the *z3* solver to find working permutations that satisfy our wishes.

This task is also challenging to extend, but the procedure would remain the same, with different constraints and a different linear structure. Future work could extend the generation of new SPN tasks to fewer or more rounds or use S-boxes of a different size.

Another possible extension would be to integrate *SageMath* and *z3* more in the tasks, increasing the performance and reducing code complexity.

The main takeaway from this thesis is that despite the help of libraries such as *SageMath* and *z3*, it is still quite a challenge to have an automated tool that guarantees the creation of new and easy-to-compute tasks for students. Since the practicals of the “Cryptography” course [IAI24b] have more assignments that students need to solve, it is still an open question to automate every task sheet.

This thesis covers two practical assignments from up to ten that students need to solve. The previous work by Romana Gruber [Gru20] covers two assignment sheets regarding *Primality* and the *Linear Feedback Shift Register (LFSR)* as well. However, in the course, there are ten assignment sheets in total.

This also means that it should be possible to extend the tool to its full extent, resulting in the opportunity for other students to work on it in the scope of their bachelor project or thesis.

Bibliography

- [AS11] Moncef Amara and Amar Siad. “Elliptic Curve Cryptography and its applications”. In: *International Workshop on Systems, Signal Processing and their Applications, WOSSPA*. 2011, pp. 247–250. DOI: 10.1109/WOSSPA.2011.5931464.
- [BCLN16] Joppe W. Bos, Craig Costello, Patrick Longa, and Michael Naehrig. “Selecting elliptic curves for cryptography: an efficiency and security analysis”. In: *Journal of Cryptographic Engineering* 6 (2016), pp. 259–286.
- [BLP15] Alex Biryukov, Gaëtan Leurent, and Léo Perrin. “Cryptanalysis of Feistel Networks with Secret Round Functions”. In: *Selected Areas in Cryptography - SAC 2015 - 22nd International Conference, Sackville, NB, Canada, August 12-14, 2015, Revised Selected Papers*. Ed. by Orr Dunkelman and Liam Keliher. Vol. 9566. Lecture Notes in Computer Science. Springer, 2015, pp. 102–121. DOI: 10.1007/978-3-319-31301-6_6.
- [Bou18] Christina Boura. *Block ciphers and Boolean functions*. 2018. URL: <https://christinaboura.files.wordpress.com/2019/11/cryptobg2018-boura.pdf> (visited on 02/14/2024).
- [BS93] Eli Biham and Adi Shamir. *Differential Cryptanalysis of the Data Encryption Standard*. Springer, 1993. ISBN: 978-1-4613-9316-0. DOI: 10.1007/978-1-4613-9314-6.
- [Cou04] Nicolas T. Courtois. “Feistel Schemes and Bi-linear Cryptanalysis”. In: *Advances in Cryptology - CRYPTO 2004, 24th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 2004, Proceedings*. Ed. by Matthew K. Franklin. Vol. 3152. Lecture Notes in Computer Science. Springer, 2004, pp. 23–40. DOI: 10.1007/978-3-540-28628-8_2.
- [Gor05] Daniel M. Gordon. “Discrete Logarithm Problem”. In: *Encyclopedia of Cryptography and Security*. Ed. by Henk C. A. van Tilborg. Boston, MA: Springer US, 2005, pp. 164–168. ISBN: 978-0-387-23483-0. DOI: 10.1007/0-387-23483-7_116.
- [Gru20] Romana Gruber. *Cryptography Task Generator*. 2020.
- [Hao11] Liu Haoru. “Elliptic Curves and Integer Factorization”. In: *University of Chicago Mathematics* (2011). URL: <https://www.math.uchicago.edu/~may/VIGRE/VIGRE2011/REUPapers/LiuH.pdf> (visited on 07/18/2024).

- [Hey02] Howard M. Heys. “A Tutorial on Linear and Differential Cryptanalysis”. In: *Cryptologia* 26.3 (2002), pp. 189–221. DOI: 10.1080/0161-110291890885. URL: <https://doi.org/10.1080/0161-110291890885>.
- [IAI24a] IAIK Institute of Applied Information Processing and Communications. *Teaching – IAIK*. 2024. URL: <https://www.iaik.tugraz.at/teaching/> (visited on 02/14/2024).
- [IAI24b] IAIK Institute of Applied Information Processing and Communications. *Cryptography – Teaching – IAIK*. 2024. URL: <https://www.iaik.tugraz.at/course/cryptography-705066-wintersemester-2023-24/> (visited on 02/14/2024).
- [IAI24c] IAIK Institute of Applied Information Processing and Communications. *Operating Systems – Teaching – IAIK*. 2024. URL: <https://www.iaik.tugraz.at/course/operating-systems-inp32512uf-sommersemester-2024/> (visited on 02/14/2024).
- [IAI24d] IAIK Institute of Applied Information Processing and Communications. *Operating Systems - TACOS Test System – Teaching – IAIK*. 2024. URL: <https://sweb.student.iaik.tugraz.at/index.php?page=exampletest> (visited on 02/14/2024).
- [LN94] Rudolf Lidl and Harald Niederreiter. *Introduction to finite fields and their applications*. Cambridge university press, 1994. ISBN: 9781139172769. DOI: <https://doi.org/10.1017/CB09781139172769>.
- [Men01] Alfred Menezes. “Evaluation of security level of cryptography: the Elliptic Curve Discrete Logarithm Problem (ECDLP)”. In: *University of Waterloo* 14 (2001). URL: <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=352185778c4b6f553092025a18ea737c8d790c72> (visited on 07/18/2024).
- [Ols79] Loren D. Olson. “The trace of frobenius for elliptic curves with complex multiplication”. In: *Algebraic Geometry*. Ed. by Knud Lønsted. Berlin, Heidelberg: Springer Berlin Heidelberg, 1979, pp. 454–476. ISBN: 978-3-540-35049-1.
- [RSA78] Ronald L. Rivest, Adi Shamir, and Leonard M. Adleman. “A Method for Obtaining Digital Signatures and Public-Key Cryptosystems”. In: *Commun. ACM* 21.2 (1978), pp. 120–126. DOI: 10.1145/359340.359342.
- [SBSB06] Sakalli, Bulus, Sahin, and Buyuksaracoglu. “Affine Equivalence in S-boxes”. In: *2006 IEEE 14th Signal Processing and Communications Applications*. 2006, pp. 1–4. DOI: 10.1109/SIU.2006.1659838.
- [Sha49] Claude E. Shannon. “Communication Theory of Secrecy Systems”. In: *Bell System Technical Journal* 28.4 (1949), pp. 656–715. DOI: 10.1002/j.1538-7305.1949.tb00928.x.

- [SSE98] S. Sutikno, A. Surya, and R. Effendi. “An implementation of ElGamal elliptic curves cryptosystems”. In: *IEEE. APCCAS 1998. 1998 IEEE Asia-Pacific Conference on Circuits and Systems. Microelectronics and Integrating Systems. Proceedings (Cat. No.98EX242)*. 1998, pp. 483–486. DOI: 10.1109/APCCAS.1998.743829.